JS Astrophysics Computational Lab: Assignment 3 Parts 1 & 2

Prof. Johanna Vos

February 17, 2026

Lab Objectives

The third and final computational lab assignment is split over 2 weeks. The goal of this lab is to further develop your python programming skills (particularly in the area of analysing large datasets) while learning about stellar astrometry and dynamics. The assignment is split into 3 parts. In the first lab you will start with Parts 1 and 2, which focus on the Gaia Catalog of Nearby Stars. In the second lab you will move onto Part 3, which focuses on sources that are missed by Gaia – very low mass stars and brown dwarfs.

Gaia provides us with the means to study the motions of stars in our Milky Way. Your scientific goal in this assignment is to identify and analyse stars and brown dwarfs that will pass, or have passed, close to the solar system, known as stellar flybys (in the case of stars) and substellar flybys (in the case of brown dwarfs). Close flybys to the solar system may have a dramatic influence on the solar system, for example by perturbing comets in the Oort Cloud. Gaia uniquely provides us with information on the position and motion of large numbers of stars in our solar neighborhood, and so provides the first opportunity to search for these flybys in a large sample.

Assignment Components

At the end of the assignment you will produce a short report that combines all three parts of the lab. This assignment is worth a third of the module and will be graded based on your report (30 marks) and source code (20 marks).

Report: Your report should follow standard lab report style as outlined in the `assignment_guidelines.pdf` document in Prof. Gibson's part of the module. You should include a title page with name and student number, an abstract, methods, results, discussion and conclusions sections. Plots should be included in the report and care should be taken to ensure that they are of scientific standard, i.e. clearly labeled, readable, adequate resolution etc. Please also include the standard signed plagiarism declaration and make sure to upload the final report as a `.pdf` file.

Source Code: Your code should be well commented so that I can clearly see the steps that you took and that you understand each step. You may submit separate python and/or notebook files (i.e. `.py` and/or `.ipynb` files).

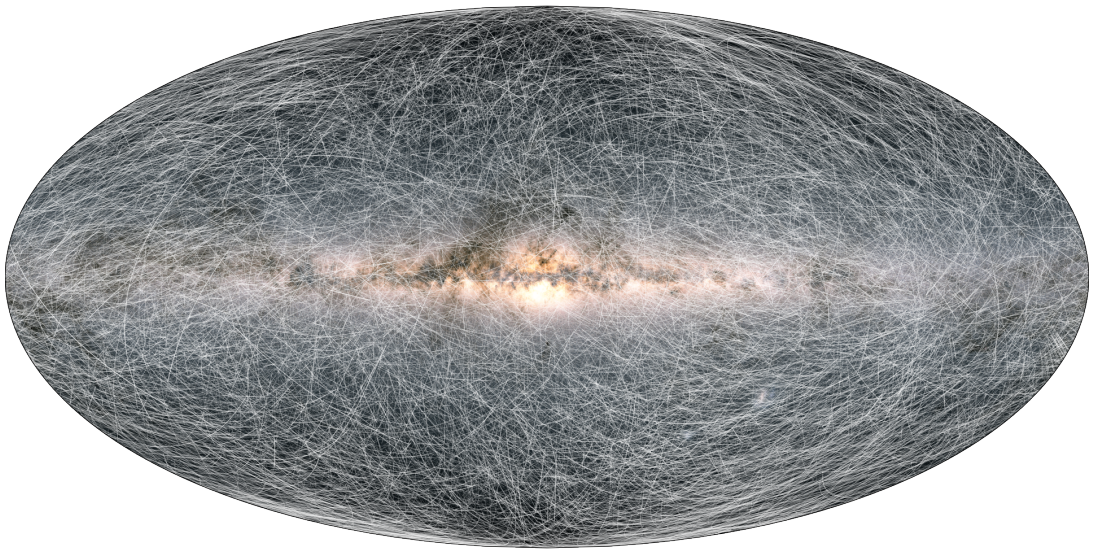


Figure 1: The stars in our galaxy are in constant motion. Gaia measures this motion – measured via *proper motion* and *radial velocity* – with unprecedented precision. The trails on this image show how 40,000 stars in the Gaia Catalog of Nearby Stars will move across the sky in the next 400,000 years. Credit: ESA/Gaia/DPAC

Submission: The report and source code should both be uploaded to Blackboard by the submission deadline of **Friday 3rd April 2026 at 5:00pm**. Late reports will be deducted 10 marks per day that they are overdue.

Stellar Flybys

The Milky Way is a highly dynamic galaxy, consisting of over 100 billion stars. Data from the Gaia mission gives us the opportunity to study the motion of approximately 2 billion stars within the Milky Way. In particular, we can use this data identify a population known as *stellar flybys* – stars that have a past or future close pass to the solar system. These close passing stars are likely to have an impact on our solar system, such as the possibility of perturbing the Oort Cloud. Perturbations in the Oort Cloud can lead comet showers and potentially even Earth extinction events (e.g. Rickman et al. 2014, Davis et al. 1984). On a more positive note, the dynamics of our solar neighborhood may bring an exoplanetary system closer to us, allowing us to study it in greater detail, and potentially to search for life.

In this assignment, you will use data from the Gaia Mission and other telescopes to identify and characterise stellar and substellar flybys in our past and future. To begin, we will use the Gaia Catalog of Nearby Stars (GCNS; Gaia Collaboration et al., 2021), which includes a census of stars out to 100 pc. Using the kinematic information (radial velocity, proper motions) and positional information (RA, Dec, parallax) provided in the GCNS, you will identify the closest approaching stars in our recent history and into the future.

The Linear Motion Approximation

The perihelion refers to the closest approach of an object to our solar system. For this assignment, the relevant parameters for each star's closest approach are the time and distance of the star at perihelion, which we will call t_{ph} and d_{ph} respectively. The time is measured with respect to the present day, and distance and speed are measured relative to the solar system barycentre. We will make the assumption that each star moves with a constant velocity, v, with respect to the Sun and follow the formulation presented in Bailer-Jones et al. 2015. Under this assumption, we can estimate t_{ph} and d_{ph} using the equations:

$$t_{ph} = -c_1 \frac{1}{\varpi} \frac{v_r}{v^2} \quad \text{(yr)} \tag{1}$$

$$d_{ph} = 10^3 \frac{1}{\varpi} \frac{v_T}{v} \quad \text{(pc)} \tag{2}$$

where ϖ is the parallax in mas, v_r is the radial velocity in kms^{-1}. Note that units are given in parentheses after each equation. Positive values of t_{ph} indicate perihelia in the future, and negative values refer to perihelia in the past. The transverse velocity, v_T, and total velocity, v, can be calculated using the equations:

$$v_T = c_2 \frac{\sqrt{\mu_{\alpha*}^2 + \mu_\delta^2}}{\varpi} \quad (\text{km s}^{-1}) \tag{3}$$

$$v = \sqrt{v_T^2 + v_r^2} \quad (\text{km s}^{-1}) \tag{4}$$

Where $\mu_{\alpha*}$ amd μ_δ are the RA and Dec proper motions in mas yr^{-1} respectively. All velocities are measured in km s^{-1} and constants c_1 and c_2 are given by:

$$c_1 = 0.97779 \times 10^9 \quad (10^3 \text{ pc km}^{-1} \text{ yr}^{-1})$$

$$c_2 = 4.74047 \quad (\text{AU km}^{-1} \text{ yr}^{-1})$$

Using these equations, we can calculate the time and distance of closest approach (t_{ph} and d_{ph}) for all the stars in the sample.

A Note on Writing Efficient Code using Vectorization

As we found out in the last lab, running calculations on large amounts of data can be time consuming. A common way of speeding up codes is to use *vectorization* in place of *for loops*. In python, loops can be slow to execute because of the overhead of the interpreter. These overheads add up when expressions are repeated many times in a loop. Vectorization replaces the loop with a single array operation and can significantly reduce the execution time of python codes. Array operations can be carried out using the `numpy` Python module. You can read more about vectorized operations at this link.

To best illustrate how to implement vectorization, we will use look at an example where we want to calculate and print the sum of a sequence of numbers between 0 and $100,000$, using loop and vectorization methods. We use the `time` module to measure how long it takes for each method:

```
1 import time
2 import numpy as np
3
```

```python
# calculate the sum using a for loop
start = time.time()
total = 0
for item in range(0, 100000):
    total = total + item
print('Sum:' + str(total))
end = time.time()
print('Computation time using loop is:' +str(end - start))

# calculate the sum using vectorization
vectorised_start_time = time.time()
print('Sum:' + str(np.sum(np.arange(100000))))
vectorised_end_time= time.time()
print('Computation time using vectorization is:' +str(
   vectorised_end_time - vectorised_start_time ))
```

Listing 1: Python example

On my laptop, I get the following output:

```
Sum:4999950000
Computation time using loop is:0.019757986068725586
Sum:4999950000
Computation time using vectorization is:0.0010459423065185547
```

Listing 2: Python output

Using vectorization, we simply applied the array operation `numpy.sum` on the entire array instead of looping through each element. The vectorized calculation runs more than 10 times faster than the for loop. As you write your code for this assignment, keep in mind that vectorized operations on `numpy` arrays will significantly speed up your computational time.

Part 1 Tasks:

1. Read in the fits file `100pc_rvnotnull.fits`. This version of the GCNS has been filtered to remove any sources without a measured radial velocity. The radial velocity and its 1σ error is stored in the `radial_velocity` and `radial_velocity_error` arrays with units of kms^{-1}. How many sources are in the sample?

2. Plot a H-R diagram for this filtered sample. Which stars are missing compared to the original GCNS catalog that we looked at in Lab 10?

3. Use the equations listed above to determine the perihelion time and distance (t_{ph} and d_{ph}) for every source in the sample. Using Monte Carlo sampling, or otherwise, ensure that you have error bars for each measurement. If using Monte Carlo sampling, plot the input and output histograms for a few sources to check if the distributions are Gaussian.

4. Make two plots of t_{ph} versus d_{ph} that shows all stellar flybys within 10 pc and within 2 pc. Add some well-known markers, such as the current distance to Proxima Centauri (our current closest stellar neighbour) and the extent of the Oort Cloud, to give a sense of scale.

5. In a timespan of 10 Myr (centred on the present day), how many sources will (or have) come closer than the distance to Proxima Centauri? How many stars will pass through the Oort Cloud?

6. Create a table to include in your report that shows the 10 closest approaching stars. Include their Gaia DR3 source ID, current distance and t_{ph} and d_{ph}. Overplot your ten closest approaching sources on the HR diagram. Can you determine their spectral types?

Estimating the Impact of Close Flybys

The closest stellar flybys may have a dramatic influence on our solar system. For example, passing stars can perturb the Oort Cloud, triggering comet showers and potentially even extinction events. The degree of perturbation, which we call the *impulse*, that comets in the Oort cloud receive depends not only of the distance of the approaching star, but also on its speed and mass. We can use a simple approximation for the impulse developed by Opik et al. 1932 and Oort et al. 1950 to estimate the relative impact of each stellar flyby. We estimate the impulse transfer by

$$I \propto \frac{M}{v_{ph} d_{ph}^2} \tag{5}$$

where M is the mass of the approaching star. In Part 1, we already calculated v_{ph} and d_{ph}, but we will need to estimate the mass of each star in order to calculate their relative impulses, and thus determine the potential impact of each stellar flyby.

Estimating Stellar Masses

Unfortunately, Gaia cannot directly "measure" the masses of stars in the GCNS sample. In fact, measuring masses in astronomy is often quite difficult. For stars, we can compare *observed* values such as magnitudes and colours to stellar models to help us to estimate *fundamental* parameters such as the effective temperature, T_{eff}, and the mass, M.

The file `Mamajek_relations.txt` contains a stellar spectral sequence, listing temperature, observed colours and masses for each spectral type. These values are based on empirical measurements and comparisons with evolutionary models (described in Pecault & Mamajek et al. 2013). In the last lab we learned to use interpolation. If we have the temperature for a particular object, we can use this table to determine its mass using interpolation. Similarly, if we have a measured colour, we can also determine its mass using interpolation. We will employ both of these methods to estimate masses for stars in our sample.

Part 2 Tasks:

1. Read in the Mamajek relation file `Mamajek_relations.txt`. It will help to open the file first with a plain text reader to see how it looks. Save the effective temperature, mass and $B_P - R_P$ columns as `numpy` arrays. You will notice '`...`' for some entries in the file – you will want to save these as `NaN`s in your arrays.

2. Plot mass versus effective temperature for the Mamajek sample. Perform an interpolation on these points to give the function $f(T_{\text{eff}}) = M$, where T_{eff} is the effective temperature and M is the mass. Use this function to determine masses for all stars in the sample with a provided T_{eff} from Gaia. Model-derived T_{eff} values are stored in the `teff_gspphot` column in the GCNS file.

3. Unfortunately, not all stars in Gaia have an estimated T_{eff}. This can be due to a number of reasons, including for binary stars, low signal-to-noise data and/or poor model fits to the spectra. You can read more about how `teff_gspphot` is calculated at this link. How many stars in the sample do *not* have an estimated T_{eff}?

4. For these stars, we can use the Mamajek relations to estimate a mass for each source in your sample based on its measured $B_P - R_P$ colour. For these stars, repeat the interpolation technique above, but this time using the empirical relation between $B_P - R_P$ and M.

5. Once you have estimated masses, calculate the impulse transfer for each source.

6. Make a new plot of t_{ph} versus d_{ph}, but this time, indicate the impulse for each source by e.g. setting the symbol size proportional to the impulse and/or using a colormap to show the impulse. Zoom in on a region showing only the closest stellar flybys. Take some time to add other components to improve the readability of your plot.

7. How does this change your view on the importance of close flybys? Which flyby is likely to have the largest impact on the solar system? Discuss its trajectory in your report.

8. Also discuss the trajectory of your favourite exoplanet-hosting star.